

FORMATION EMBARQUÉE

Évaluation pratique — Arduino

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Évaluation pratique — Arduino (2 h, sur Wokwi)

Épreuve **entièrement sur simulateur** : <https://wokwi.com> (Arduino Uno). Documents autorisés.

Rendu : le lien de partage du projet Wokwi (bouton *Share*) — l'examineur DOIT pouvoir cliquer ► et dérouler la recette.

Sujet — Minuterie d'escalier améliorée

Monter dans Wokwi : 1 bouton (D2), 1 LED « éclairage » (D9, PWM), 1 LED témoin rouge (D8), 1 potentiomètre (A0).

Comportement exigé : 1. Appui court → éclairage **plein feu** pendant une durée réglée par le potentiomètre (5 à 60 s, relue à chaque appui). (/4) 2. Les **10 dernières secondes**, l'éclairage « prévient » : luminosité réduite à 30 % (PWM), témoin rouge clignotant à 2 Hz. (/4) 3. Un appui pendant la marche **relance** la durée complète (sans à-coup visible). (/2) 4. **Appui long (≥ 2 s)** → éclairage permanent ; nouvel appui long → retour au mode minuterie. (/4) 5. Moniteur série 115200 : une ligne par changement d'état (**MARCHE 42s** , **PREAVIS** , **ARRET** , **PERMANENT**). (/2)

Contraintes de plateforme (éliminatoires si violées)

- **Aucun** `delay()` dans `loop()` — tout au `millis()` . Le correcteur met un appui pendant le préavis : le clignotant ne doit pas « geler ».
- Anti-rebond logiciel du bouton (20 ms) — Wokwi simule des rebonds si on active *bounce* dans les propriétés du bouton : **l'activer**.
- Architecture : une FSM explicite (`enum` + `switch`) — le correcteur lit le code : des `if` imbriqués sans états nommés = -4.

Recette déroulée par le correcteur (10 min)

#	Action	Attendu
1	► puis appui court, pot à fond	plein feu, série MARCHE 60s
2	attendre le préavis	30 % + témoin 2 Hz + PREAVIS
3	appui pendant le préavis	retour plein feu, durée relancée
4	appui long	PERMANENT , plus de minuterie
5	appui long à nouveau	retour minuterie, ARRET

Barème

Fonctionnel (ci-dessus) /16 · qualité (FSM lisible, pas de globales inutiles, noms clairs) /2 · robustesse (rebonds activés, pas de gel) /2. **Seuil : 14/20**. Bonus +1 : la durée restante affichée chaque seconde sans spammer (une ligne/s exactement).

FORMATION EMBARQUÉE

Évaluation pratique — C / C++

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Évaluation pratique — C / C++ (2 h, sur PC)

Épreuve sur machine, **documents autorisés** (le vrai métier se fait docs ouvertes) mais **zéro copier-coller** depuis les corrigés. Plateforme : GCC/G++ local ou <https://www.onlinegdb.com>. Rendu : les fichiers sources + un fichier **REPONSES.md** pour les questions.

Partie A — Questions flash (20 min, /4)

Répondre en 1 à 3 phrases, sans compiler : 1. Que vaut `uint8_t x = 200; x += 100;` et pourquoi ? (/1) 2. `volatile` garantit-il l'atomicité ? Justifier avec un exemple 16 bits sur CPU 8 bits. (/1) 3. Pourquoi `#define CARRE(x) x*x` est-il faux ? Donner l'appel qui le piège et la correction complète. (/1) 4. C++ : pourquoi un destructeur virtuel quand on détruit via un pointeur de base ? Que se passe-t-il sans ? (/1)

Partie B — Programmation C (60 min, /10)

Écrire `histogramme.c` : un module qui reçoit des octets un par un (`void histo_ajouter(uint8_t v)`) et les classe dans 8 tranches de 32 (`0-31` , `32-63` , ...). Fournir : - `histo_init()` , `histo_ajouter()` , `uint32_t histo_tranche(uint8_t n)` ; - le calcul de la tranche **par décalage** (pas de division ni de `if` en cascade) (/3) ; - un `main` de test avec ≥ 5 `assert` couvrant les bords (0, 31, 32, 255, tranche invalide) (/3) ; - compilation **sans aucun warning** avec `-Wall -Wextra` (/2) ; - pas de variable globale accessible de l'extérieur (`static`) (/2).

Partie C — Lecture critique C++ (40 min, /6)

Le code suivant compile. Lister **quatre** défauts du point de vue « C++ embarqué », corriger chacun (code + une phrase), du plus grave au moins grave :

```
class Journal {
public:
    Journal() { buf = new char[256]; }
    void log(std::string m) { lignes.push_back(m); }
    char* buf;
    std::vector<std::string> lignes;
};
Journal j1, j2 = j1;
```

(pistes : fuite, copie, passage par valeur, croissance non bornée, visibilité des membres...)

Barème et seuil

Partie	Points
A — questions flash	/4
B — module C testé	/10
C — lecture critique	/6
Total (seuil : 14/20)	/20

Spécificité de l'épreuve C/C++ : on note la **robustesse aux bords** et la **propreté de compilation**, pas la vitesse d'écriture. Un module B sans les asserts de bord plafonne à 7/10 même s'il « marche ».

FORMATION EMBARQUÉE

Évaluation pratique — Java

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Évaluation pratique — Java (2 h, sur PC)

JDK $\geq$ 17, IDE libre (ou <https://www.onlinegdb.com> pour dépanner). Documents autorisés. Rendu : les `.java` + sortie console des tests. Spécificité Java : on évalue la **modélisation objet** et la **concurrency propre** — pas du C déguisé en Java.

Sujet — Passerelle de mesures

Une passerelle reçoit des trames binaires de capteurs et les expose sous forme d'objets.

Partie A — Modèle objet (/7)

- `record` `Mesure(int idCapteur, double valeur, long horodatageMs)` ;
- interface `Decodeur` : `Optional<Mesure> decoder(byte[] trame);` ;
- deux implémentations : `DecodeurTemperature` (trame `{0xA1, id, hi, lo}`, valeur = mot 16 bits **signé** $\div 10$) et `DecodeurHumidite` (trame `{0xA2, id, v}`, valeur = octet 0..100) ;
- une `Passerelle` qui reçoit un `byte[]`, choisit le bon décodeur d'après l'octet de tête, et renvoie `Optional<Mesure>`.

Points sensibles notés : les masques `& 0xFF` partout (/2), le mot **signé** de la température (contrairement au mini-TP : `(short)` cast à justifier en commentaire) (/2), l'ajout d'un 3^e décodeur **sans modifier** `Passerelle` (expliquer en commentaire comment) (/1), rejets propres via `Optional` — aucune exception pour une trame invalide (/2).

Partie B — Concurrency (/8)

- Un thread « producteur » qui pousse des trames simulées (5/s) dans une `BlockingQueue<byte[]>` **bornée à 32** ;
- un thread « consommateur » qui décode et affiche ;
- arrêt propre des deux threads après 5 s via `interrupt()` + `join()` (le `main` se termine sans `System.exit()`) (/3) ;
- la file pleine ne perd pas silencieusement : choisir bloquer OU compter les rejets, et l'écrire en commentaire (/2) ;
- statistiques finales : nombre produit / consommé / rejeté cohérents (/3).

Partie C — Question d'architecture (/5)

En 15 lignes max dans `REPONSES.md` : cette passerelle doit maintenant **historiser en base et exposer un endpoint HTTP**. Découpage en classes/threads proposé, et : pourquoi le callback réseau ne doit-il jamais écrire en base directement ? (*attendu : file + thread dédié — la règle « ISR courte » version serveur, TD 05 ex. 5.*)

Barème

A / 7 · B / 8 · C / 5 — **seuil : 14/20**. Éliminatoire : un `catch (InterruptedException)` vide (le drapeau doit être restauré ou le thread doit sortir).

FORMATION EMBARQUÉE

Évaluation pratique — Schneider

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Évaluation pratique — Schneider EcoStruxure (2 h 30, simulateur M221)

Machine Expert Basic (gratuit) + simulateur intégré ; client Modbus au choix (`mbpoll` , `pymodbus` , QModMaster). Documents autorisés. Rendu : projet `.smbp` + **document de table Modbus** + journal du client PC. Spécificité Schneider de cette épreuve : l'ouverture **Modbus** vers la supervision fait partie du sujet — c'est la marque de fabrique de l'écosystème (cours 08 §5).

Sujet — Ventilation de parking

Deux ventilateurs V1/V2 (`%Q0.0` , `%Q0.1`) pilotés par un capteur de CO analogique (`%IW0.0` , 0..1000 = 0..300 ppm) :

- CO > 100 ppm → **un** ventilateur (alternance à chaque démarrage selon les heures de marche — reprise du TD 08) ;
- CO > 200 ppm → **les deux** ;
- hystérésis de 20 ppm sur chaque seuil (pas de battement) ;
- capteur figé 60 s pendant qu'un ventilateur tourne → défaut capteur → **les deux ventilateurs en marche** (repli sécuritaire : on ventile en aveugle) + voyant `%Q0.2` .

Travail demandé et barème

#	Livrable	Points
1	Mise à l'échelle + symboles complets AVANT le code	/2
2	Double hystérésis correcte (recette : monter/descendre le CO au simulateur et cocher les 8 points de bascule attendus)	/5
3	Alternance par compteurs d'heures (<code>%MW</code>), choix figé au front	/4
4	Chien de garde capteur + repli « tout ventiler » justifié en commentaire	/3
5	Table Modbus documentée (≥ 6 mots : état bits, CO, heures V1/V2, mot de vie, commande) + zones disjointes + bornage des écritures PC	/4
6	Preuve client PC : lecture cyclique + détection du mot de vie figé quand on stoppe le simulateur (capture/journal)	/2

Seuil : 14/20. Éliminatoire : une écriture Modbus du PC qui pilote directement une `%Q` (le PC écrit des demandes, l'automate décide).

Question orale (ajuste ±2)

« Pourquoi le repli capteur est-il *tout ventiler* ici, alors que le thermostat du TD 03 coupait le chauffage en panne capteur ? » — (attendu : l'état sûr dépend du procédé — *sur-ventiler un parking est sans danger*,

sous-ventiler tue ; sur-chauffer une pièce est dangereux, sous-chauffer non. L'état de repli se déduit du risque, pas d'une règle mécanique.)

FORMATION EMBARQUÉE

Évaluation pratique — Siemens

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Évaluation pratique — Siemens TIA Portal (3 h, sur PLCSIM)

Épreuve **sans automate** : TIA Portal + PLCSIM. Documents autorisés. Rendu : l'archive projet TIA + la **recette de test remplie et datée** + captures des tables de visualisation aux moments clés. Spécificité automate : la **démarche** (E/S → GRAFCET → blocs → recette) est notée autant que le fonctionnement — un programme qui marche sans dossier d'analyse plafonne à 10/20.

Sujet — Portail coulissant automatique

Un portail motorisé : ouverture/fermeture par moteur 2 sens (`%Q0.0` ouvre, `%Q0.1` ferme), fins de course ouvert/fermé (`%I0.2` , `%I0.3`), cellule de sécurité (`%I0.4` , **NF** : 1 = passage libre), badge (`%I0.0`), arrêt d'urgence (`%I0.1` , NF).

Comportement : badge → ouverture complète → tempo 10 s → fermeture auto. Pendant la fermeture, cellule coupée → **réouverture immédiate**. Défaut « moteur » si un mouvement dure > 15 s sans atteindre sa fin de course (surveillance de mouvement). AU et réarmement selon les règles du TD 07.

Travail demandé et barème

#	Livable	Points
1	Avant tout code : liste d'E/S (avec NO/NF justifiés) + GRAFCET papier/photo	/4
2	Structure : <code>FB_Sequence</code> (SCL, CASE fidèle au GRAFCET), <code>FB_Defaults</code> , <code>FC_Sorties</code> seul écrivain des %Q , verrouillage croisé ouvre/ferme	/5
3	Surveillance de mouvement 15 s (TON) + position de repli argumentée en commentaire (le portail s'arrête ? rouvre ?)	/3
4	Recette écrite PUIS déroulée sous PLCSIM : ≥ 8 scénarios dont AU en mouvement, cellule en fermeture, badge pendant fermeture, défaut moteur	/5
5	Réarmement conforme : disparition cause ET acquit — testé (scénario dédié)	/2
6	Table de visualisation propre (étape, E/S, tempos) — capture	/1

Seuil : 14/20. Éliminatoires : les deux sorties moteur actives ensemble (à n'importe quel instant de n'importe quel scénario), ou une sécurité implémentée dans la séquence au lieu d'en aval.

Question orale (ajuste ±2)

« La cellule tombe en panne débranchée (fil coupé). Que fait ton portail ? » — la réponse doit découler du choix NF de la liste d'E/S, pas d'une réflexion improvisée.

FORMATION EMBARQUÉE

Évaluation pratique — Stm32

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Évaluation pratique — STM32 (3 h, Nucleo ou justification simulateur)

STM32CubeIDE + Nucleo-F411 (ou équivalente). Sans carte : parties A et C intégralement faisables (calculs + code + debug en « simulation de raisonnement ») — la partie B se rend alors en code commenté avec les valeurs de registres attendues. Documents autorisés. Rendu : projet CubeIDE + **REPONSES.md** + captures du débogueur. Spécificité STM32 : on note la **maîtrise des registres et du débogueur**, pas l'assemblage de bibliothèques.

Partie A — Calculs de configuration (30 min, /5)

SYSCLK = 84 MHz, APB1 timer clock = 84 MHz. Sans outil, calculatrice autorisée : 1. PSC et ARR pour une interruption TIM3 toutes les **50 ms** (deux couples possibles — en donner un et vérifier). (/2) 2. Baudrate réel et erreur en % pour un UART réglé « 115200 » si le diviseur retenu donne 84 MHz / 729. (/2) 3. L'ADC 12 bits lit 2482 avec Vref = 3,3 V : tension ? Et en **point fixe** millivolts sans flottant (formule entière) ? (/1)

Partie B — Réalisation (90 min, /10)

Sur Nucleo : un « chenillard de charge » — la LED LD2 clignote à une fréquence proportionnelle à la valeur du potentiomètre (ou du capteur de température interne si pas de potentiomètre) :

Exigence	Points
Clignotement cadencé par interruption TIM3 (pas de HAL_Delay), période recalculée depuis l'ADC	/3
ADC lu par DMA circulaire — preuve au débogueur : tableau qui bouge CPU arrêté (capture Live Expressions)	/3
Console UART : status renvoie période courante + valeur ADC + uptime ; réception par interruption, réarmée	/3
printf redirigé, bannière avec __DATE__ / __TIME__	/1

Partie C — Diagnostic (60 min, /5)

Le correcteur fournit (ou tu écris toi-même) un projet contenant **trois bugs classiques** : horloge RCC d'un GPIO non activée, variable partagée ISR/main sans **volatile**, **HAL_UART_Receive_IT** non réarmé.

Pour chacun : symptôme observé, méthode de localisation (quel outil du débogueur ?), correction, et **la règle générale** en une phrase. (/1,5 par bug + /0,5 pour la qualité de la démarche décrite.)

Barème

A /5 · B /10 · C /5 — **seuil : 14/20**. La capture « DMA qui bouge CPU arrêté » est le jalon central de B : sans elle, B plafonne à 5/10, car c'est la preuve que le DMA est compris et pas recopié.

FORMATION EMBARQUÉE

Évaluation pratique — Vhdl

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Évaluation pratique — VHDL (2 h 30, sur simulateur)

Épreuve sur <https://www.edaplayground.com> (GHDL) ou GHDL local. Documents autorisés. Rendu : les `.vhd` + le **journal de simulation** (copie de la console) + 2 captures de chronogrammes annotées. Spécificité VHDL : **le testbench vaut autant que le design** — c'est la discipline du métier FPGA.

Sujet — Générateur d'impulsion calibrée (« one-shot »)

Concevoir `oneshot` : sur un front montant de `declencher`, la sortie `pulse` passe à '1' pendant **exactement N cycles** (N = generic, défaut 10), puis retombe. Les re-déclenchements pendant l'impulsion sont **ignorés** (non re-déclenchable).

```
entity oneshot is
    generic ( N : natural := 10 );
    port ( clk, declencher : in std_logic;
          pulse : out std_logic );
end entity;
```

Travail demandé et barème

#	Livrable	Points
1	Détecteur de front interne (2 bascules — <code>declencher</code> est déjà synchrone, on ne demande PAS le synchroniseur)	/3
2	FSM ou compteur : <code>pulse</code> haut N cycles pile (vérifié au curseur)	/5
3	Non-redéclenchement : un 2 ^e front pendant l'impulsion ne prolonge rien	/3
4	Testbench auto-vérifiant : ≥ 4 <code>assert</code> (largeur exacte, repos avant/après, re-déclenchement ignoré, second tir possible après retombée)	/6
5	Chronogramme annoté : largeur mesurée aux curseurs + retour ligne	/2
6	Zéro <code>wait for</code> dans le design (simulation seulement), zéro latch	/1

Seuil : 14/20. Éliminatoire : un design qui ne passe pas son propre testbench, ou un testbench qui ne teste que le cas nominal.

Questions orales de soutenance (5 min, ajustent ± 2)

1. Ton compteur compte de 0 à N-1 ou de 1 à N ? Montre au chronogramme pourquoi ta borne donne exactement N cycles.
2. Que synthétise ton détecteur de front ? (dessine les 2 bascules + la porte).

3. Si **de**clencher venait d'un bouton physique, que faudrait-il ajouter et pourquoi ? (*synchroniseur + anti-rebond — cours 04 §4.2/TD 04.*)

FORMATION EMBARQUÉE

Projets d'évaluation

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Projets d'évaluation finale (avec barème)

Trois sujets « examen », un par grande voie. Choisis-en **un** en fin de parcours (ou fais les trois à quelques mois d'écart). Conditions : partir de zéro, documenter comme si un collègue devait reprendre le projet, et tenir un journal de bord (une entrée par session de travail). Durée indicative : 30-40 h chacun.

Sujet A — Firmware : enregistreur de données autonome (STM32)

Cahier des charges

Un enregistreur de mesures environnementales sur batterie :

1. Mesure T°/humidité/pression (BME280, driver **maison**) toutes les période configurables (10 s à 1 h).
2. Horodatage par **RTC interne** du STM32 (réglable par la console).
3. Stockage en **flash SPI externe ou EEPROM I2C** (driver maison aussi), format binaire compact documenté, à l'épreuve des coupures (un enregistrement à moitié écrit ne corrompt pas les autres).
4. **Console série** : `status` , `dump [n]` (relit les n derniers enregistrements en CSV), `set period` , `set time` , `erase` (avec confirmation).
5. **Basse consommation** : mode Stop entre deux mesures, réveil par RTC ; mesurer et documenter la consommation estimée (fiche de calcul d'autonomie sur pile CR2032 ou batterie LiPo).
6. Robustesse : capteur débranché, flash pleine (politique circulaire documentée), montre jamais réglée — tout est géré et signalé.

Barème (/100)

Critère	Points
Drivers BME280 + mémoire : propres, sans dépendance, erreurs gérées	20
Format de stockage documenté + intégrité sur coupure (test à l'appui)	15
Console complète et robuste (entrées hostiles testées)	10
Basse consommation mise en œuvre + calcul d'autonomie argumenté	15
Architecture (modules, zéro accès matériel hors drivers, FSM claires)	15
Qualité C : -Wall -Wextra propre, types stdint, pas de malloc, asserts	10
Git (historique lisible) + README (schéma, photos, mode d'emploi)	10
Journal de bord honnête (impasses comprises)	5

Seuil de réussite : 70. Excellence (90+) : ajout d'un watchdog documenté, tests unitaires des drivers compilés sur PC (mock du bus I2C).

Sujet B — FPGA : chronomètre d'affichage multiplexé (VHDL)

Cahier des charges

Sur simulateur (GHDL) puis carte si disponible :

1. Chronomètre MM:SS.CC (centièmes), boutons start/stop/reset (anti-rebond **matériel** du TD 04).
2. Affichage sur **4 afficheurs 7 segments multiplexés** (rafraîchis à ~1 kHz par balayage — le module de multiplexage est un composant réutilisable avec generics).
3. **Mémoire de 4 temps intermédiaires** (bouton « lap ») relus par un 5^e bouton, avec indication du numéro affiché.
4. Sortie **UART TX** (ton module du TP 2) : chaque « lap » émet le temps en ASCII (`01:23.45\r\n`).
5. Toute la conception **synchrone à une seule horloge**, entrées synchronisées, zéro latch (le rapport de synthèse en atteste si carte).

Barème (/100)

Critère	Points
Découpage hiérarchique (diviseurs, compteurs BCD en cascade, mux 7 seg, FSM boutons, UART)	20
Testbenchs : un par module + un testbench top avec scénario complet	25
Zéro latch, zéro signal multi-piloté, synchroniseurs présents	15
Chronomètre juste (vérifié au chronogramme : 1 centième = N cycles exact)	10
Laps : mémorisation/relecture correctes (cas « 5 ^e lap » défini et testé)	10
UART conforme (trame vérifiée au chronogramme)	10
Compte rendu : schéma bloc, chronogrammes annotés, journal	10

Seuil : 70. Excellence : contraintes + passage sur carte réelle avec photo, ou ajout d'un réglage de l'heure via UART RX.

Sujet C — Automatisme : machine de conditionnement (TIA Portal ou EcoStruxure)

Cahier des charges

Une machine emballe des produits par lots :

1. Convoyeur d'amenée, détection produit, **poussoir** à vérin (capteurs sorti/rentre) qui transfère le produit vers la zone de cartonnage.
2. Un carton = N produits (N réglable 4-24). Carton plein → tapis d'évacuation 5 s → nouveau carton (signal « carton présent » requis).
3. Modes AUTO / MANU (commandes unitaires sécurisées) / **REGLAGE** (vérin pas à pas, réservé au niveau « régleur » de l'HMI).

4. Défauts gérés : AU (NF), vérin qui n'atteint pas son capteur en 3 s (**surveillance de mouvement**), absence carton, bourrage cellule. Chaque défaut : arrêt sûr adapté (le vérin **ne revient pas tout seul** si un produit est engagé — à justifier), voyant, alarme HMI, acquit.
5. HMI : conduite (synoptique animé, compteurs lot/total), réglages (bornés, par niveau), page alarmes avec historique.
6. **Table Modbus/OPC documentée** (10 mots min. avec mot de vie) + un client PC (Python/Java) qui journalise la production en CSV.

Barème (/100)

Critère	Points
Dossier d'analyse AVANT code : E/S, GRAFCET (conduite + surveillance), analyse de défauts	25
Structure blocs : séquence / défauts / modes / sorties séparés, sorties centralisées	15
Surveillances de mouvement et positions de repli justifiées	15
Recette de test écrite et déroulée (≥ 10 scénarios dont 4 de défaut)	15
HMI complète (impulsions, bornes, niveaux, alarmes)	10
Table d'échange + client PC avec mot de vie	10
Journal + captures du déroulé de recette	10

Seuil : 70. Excellence : la même application portée sur le **deuxième** environnement (TIA ↔ EcoStruxure) avec une note comparant les deux — c'est un excellent sujet d'entretien.

Comment t'auto-évaluer honnêtement

1. Note chaque ligne du barème **avec une preuve** (capture, extrait de code, ligne du journal). Pas de preuve = pas les points.
2. Laisse reposer une semaine, puis rejoue TA recette de test depuis zéro (environnement fraîchement ouvert). Ce qui casse ta note mieux que toi.
3. Publie le projet sur GitHub avec le barème rempli dans le README : l'exercice de transparence est en soi une compétence professionnelle — et un excellent signal en entretien.

FORMATION EMBARQUÉE

QCM de validation

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

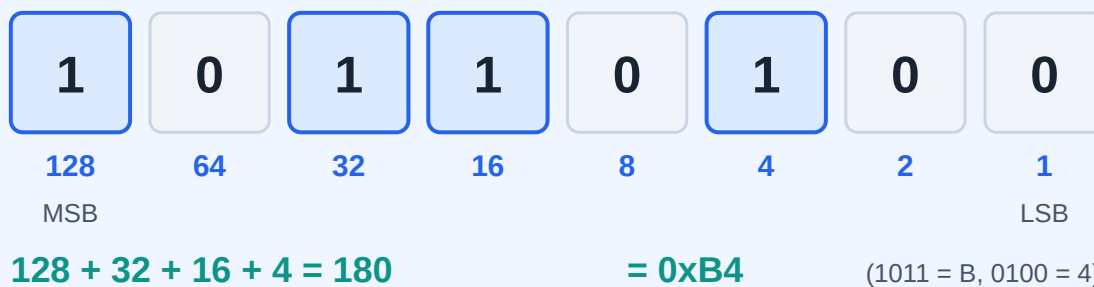
QCM de validation par module

Règle : fais le QCM d'un module **sans notes**, après le cours et le TD. Objectif : $\geq 80\%$ avant de passer au module suivant. Les corrigés (avec justification) sont en bas de page — ne triche que contre toi-même.

Module 00 — Fondamentaux (10 questions)

Aide-mémoire visuel pour les Q1-Q2 :

Nombre binaire 1011 0100 sur 8 bits



Poids des bits et conversion hexa

- Q1.** `0x3C` vaut en décimal : a) 54 b) 60 c) 66 d) 74
- Q2.** Sur 8 bits en complément à deux, `0xFF` représente : a) 255 b) -1 c) -127 d) -128
- Q3.** Une bascule D recopie D sur Q : a) en permanence b) quand l'horloge est haute c) au front d'horloge d) à la mise sous tension
- Q4.** Le bus qui exige une ligne de sélection (CS) par esclave : a) UART b) I2C c) SPI d) CAN
- Q5.** Les pull-ups sont indispensables sur : a) UART b) I2C c) SPI d) CAN
- Q6.** Une ISR doit être courte parce que : a) la pile est petite b) elle bloque le programme principal et potentiellement d'autres interruptions c) le compilateur l'impose d) elle s'exécute plus lentement
- Q7.** `volatile` garantit : a) l'atomicité b) la relecture à chaque accès c) la protection contre les ISR d) l'alignement
- Q8.** Un PWM à rapport cyclique 25 % sur une LED : a) l'allume 1 s sur 4 b) réduit sa luminosité apparente à ~25 % c) réduit sa tension à 25 % en continu d) la fait clignoter visiblement
- Q9.** Dans la toolchain, l'éditeur de liens (linker) : a) traduit le C en assembleur b) résout les adresses entre fichiers objets c) flashe la puce d) supprime les commentaires
- Q10.** « Temps réel » signifie : a) très rapide b) délai de réponse garanti c) sans OS d) multitâche

Module 01 — C (10 questions)

- Q11.** `x |= (1 << 3);` : a) teste le bit 3 b) met le bit 3 à 1 c) met le bit 3 à 0 d) inverse le bit 3
- Q12.** `uint8_t i = 0; i--;` donne : a) -1 b) 0 c) 255 d) indéfini
- Q13.** Si `p` est un `uint32_t*`, `p + 1` avance de : a) 1 octet b) 2 octets c) 4 octets d) 8 octets
- Q14.** Retourner l'adresse d'une variable locale est : a) valide b) valide si static c) un comportement indéfini d) une erreur de compilation — (deux réponses à discuter)
- Q15.** `const uint8_t *p` signifie : a) pointeur constant b) données pointées non modifiables via p c) les deux d) rien de spécial
- Q16.** En embarqué critique, `malloc` après l'init est : a) recommandé b) obligatoire c) proscrit (fragmentation/imprévisibilité) d) plus rapide
- Q17.** Une variable partagée ISR/main doit être : a) globale b) static c) volatile (et protégée si > taille atomique) d) const
- Q18.** `#define MIN(a,b) a < b ? a : b` est dangereux car : a) trop lent b) non parenthésé (précédence) c) interdit en C99 d) réentrant
- Q19.** `sizeof` d'une struct peut dépasser la somme de ses champs à cause : a) du compilateur qui se trompe b) du padding d'alignement c) des pointeurs d) de l'endianness
- Q20.** Le `break` oublié dans un `switch` : a) erreur de compilation b) le cas suivant s'exécute aussi c) sortie du switch d) boucle infinie

Modules 02-03 — C++ / Arduino (10 questions)

- Q21.** RAI signifie que la ressource est libérée : a) par le GC b) par le destructeur, automatiquement c) par `free()` d) à la fin du programme
- Q22.** Un destructeur virtuel est nécessaire quand : a) toujours b) on détruit via un pointeur de classe de base c) la classe a des templates d) jamais en embarqué
- Q23.** Le polymorphisme par templates par rapport aux fonctions virtuelles : a) est résolu à l'exécution b) est résolu à la compilation (zéro indirection) c) consomme plus de RAM d) est plus lent
- Q24.** Sur Arduino Uno, `int` fait : a) 8 bits b) 16 bits c) 32 bits d) 64 bits
- Q25.** `millis() - t0 >= periode` plutôt que `millis() >= t0 + periode` : a) plus lisible b) robuste au débordement de `millis()` c) plus rapide d) identique
- Q26.** `INPUT_PULLUP` sur un bouton vers GND : appui lu comme : a) HIGH b) LOW c) flottant d) analogique
- Q27.** Dans une ISR Arduino, `Serial.println` est : a) conseillé pour déboguer b) interdit/dangereux c) sans effet d) plus rapide qu'en boucle
- Q28.** `analogWrite(pin, 128)` produit : a) 2,5 V continu b) un PWM à ~50 % c) la moitié du courant d) une erreur
- Q29.** La classe `String` d'Arduino sur Uno pose problème car : a) trop lente b) fragmentation du tas (2 Ko de RAM) c) pas de méthodes d) non portable

Q30. `Led(const Led&) = delete;` sert à : a) détruire l'objet b) interdire la copie c) économiser la flash d) rendre la classe abstraite

Module 04 — VHDL (8 questions)

Q31. Le VHDL décrit : a) des instructions séquentielles b) un circuit dont tout fonctionne en parallèle c) un bytecode d) des scripts

Q32. Deux process qui écrivent le même signal : a) le dernier gagne b) conflit « multiple drivers » c) moyenne d) autorisé avec resolved types seulement — (b sauf `std_logic` résolu : à discuter)

Q33. Un process combinatoire qui n'affecte pas sa sortie dans toutes les branches crée : a) une erreur b) un latch involontaire c) une bascule d) rien

Q34. `signal` vs `variable` dans un process : le signal prend sa valeur : a) immédiatement b) à la fin du delta-cycle c) au reset d) aléatoirement

Q35. `wait for 10 ns` est : a) synthétisable b) simulation seulement c) synthétisable sur Xilinx d) obligatoire

Q36. Une entrée asynchrone (bouton) doit passer par : a) un pull-up b) 2 bascules de synchronisation c) un latch d) rien

Q37. En UART, le premier bit de données transmis est : a) le MSB b) le LSB c) la parité d) le stop

Q38. `unsigned` (`numeric_std`) plutôt que `std_logic_vector` pour : a) les E/S top-level b) tout ce qui compte/calcul c) les horloges d) les resets

Modules 07-08 — Automatismes (12 questions)

Q39. Le cycle automate est : a) événementiel b) lire entrées → exécuter → écrire sorties c) multitâche préemptif d) aléatoire

Q40. Un arrêt d'urgence se câble en : a) NO b) NF (sécurité positive) c) analogique d) réseau

Q41. `%Qw`, `%I0.3`, `%MW100` (Siemens) sont : a) sortie mot / entrée bit / mot mémoire b) tous des bits c) tous des mots d) des DB

Q42. Un FB diffère d'une FC par : a) le langage b) sa mémoire propre (DB d'instance) c) la vitesse d) rien

Q43. Un TON : a) retarde l'enclenchement b) retarde le déclenchement c) compte des pièces d) génère du PWM

Q44. Dans un FB appelé chaque cycle, compter des appuis sans détection de front : a) fonctionne b) compte des centaines de fois par appui c) erreur de compilation d) plus précis

Q45. La logique de maintien d'un bouton HMI appartient : a) à l'écran b) à l'automate c) au réseau d) au variateur

Q46. Modbus : les holding registers se lisent avec la fonction : a) 01 b) 02 c) 03 d) 05

Q47. Le « mot de vie » sert à : a) compter les heures b) détecter un automate figé malgré une liaison ouverte c) chiffrer d) synchroniser l'horloge

Q48. SCL/ST : l'affectation s'écrit : a) `=` b) `:=` c) `<=` d) `==`

Q49. En GRAFCET, une transition est franchie si : a) l'étape amont est active ET la réceptivité vraie b) la réceptivité est vraie c) l'opérateur appuie d) le cycle est fini

Q50. Une coupure de sécurité (porte ouverte → moteur coupé) vit : a) dans la séquence SFC b) en aval des sorties, hors séquence c) dans l'HMI d) dans la supervision

Module 10 — STM32 (6 questions)

Q51. Périphérique qui « ne répond pas » : premier réflexe : a) changer de carte b) vérifier l'horloge RCC du périphérique c) réinstaller CubeIDE d) baisser la fréquence

Q52. $f_{PWM} = f_{timer} / ((PSC+1)(ARR+1))$: pour 1 kHz à 100 MHz avec ARR=999, PSC = : a) 9 b) 99 c) 999 d) 100

Q53. La HAL I2C attend l'adresse : a) 7 bits telle quelle b) décalée d'un bit à gauche c) sur 10 bits d) en BCD

Q54. Le DMA sert à : a) déboguer b) transférer mémoire ↔ périphérique sans CPU c) accélérer l'horloge d) protéger la flash

Q55. Ne pas réarmer `HAL_UART_Receive_IT` dans le callback → a) rien b) on ne reçoit que le premier octet c) débordement d) HardFault

Q56. `BFAR` après un BusFault contient : a) le code d'erreur b) l'adresse mémoire fautive c) la ligne de code d) le numéro de tâche

Corrigé (avec justifications brèves)

Q	Rép.	Justification
1	b	$3 \times 16 + 12 = 60$
2	b	tous bits à 1 = -1 en complément à 2
3	c	basculer D = échantillonnage au front
4	c	SPI : un CS par esclave ; I2C adresse, UART point à point
5	b	I2C est à drain ouvert : sans pull-up, pas de niveau haut
6	b	elle préempte tout ; longue = latences et pertes d'événements
7	b	volatile = pas de mise en cache ; PAS d'atomicité (Q17)
8	b	puissance moyenne ; à quelques kHz l'œil intègre
9	b	le linker fixe les adresses et lie les .o
10	b	garantie d'échéance, pas vitesse
11	b	OR avec un masque = mise à 1
12	c	non signé : arithmétique modulo 256
13	c	arithmétique de pointeur en unités d'élément (4 octets)
14	c	l'objet est détruit ; (b) rend l'adresse valide mais change la sémantique
15	b	lire de droite à gauche : pointeur vers const uint8_t
16	c	fragmentation + échec imprévisible (MISRA)
17	c	et section critique si multi-octets sur petit CPU
18	b	MIN(x, y) + 1 etc. se parenthèse mal ; toujours ((a)<(b)?(a):(b))
19	b	alignement des champs
20	b	fallthrough silencieux
21	b	destructeur appelé à la sortie de portée, même sur return anticipé
22	b	sinon le destructeur dérivé n'est pas appelé (fuite/UB)
23	b	monomorphisation à la compilation
24	b	AVR 8 bits : int = 16 bits
25	b	la soustraction non signée absorbe le wrap de 49 j
26	b	pull-up interne : repos=HIGH, appui=LOW
27	b	Serial utilise des IRQ/tampons : blocage possible
28	b	$128/255 \approx 50\%$ de rapport cyclique

Q	Rép.	Justification
29	b	allocations dynamiques répétées dans 2 Ko
30	b	interdit la copie (ex. objet RAI, driver)
31	b	description matérielle, parallélisme intrinsèque
32	b	multiple drivers = erreur (sauf tri-state voulu avec 'Z')
33	b	il faut mémoriser → latch inféré
34	b	c'est ce qui modélise la bascule
35	b	le temps n'existe pas en synthèse
36	b	synchroniseur anti-métastabilité
37	b	LSB first
38	b	unsigned sait compter ; slv pour les interfaces neutres
39	b	cycle scan, mémoire image
40	b	fil coupé = déclenchement (sécurité positive)
41	a	Q=sortie, I=entrée, M=mémoire ; W=mot, x.y=bit
42	b	FB + DB d'instance ≈ classe + objet
43	a	Timer ON delay
44	b	le FB tourne à chaque cycle (~ms)
45	b	liaison HMI perdue ≠ ordre bloqué ; l'automate décide
46	c	FC03 read holding ; 01=coils, 05=write coil
47	b	compteur qui doit bouger : liaison OK ≠ programme vivant
48	b	<code>=</code> est la comparaison en ST
49	a	les deux conditions, toujours
50	b	une séquence peut rester bloquée ; la sécurité est transversale
51	b	RCC : l'oubli n°1
52	b	$100e6/((99+1)(999+1)) = 1 \text{ kHz}$
53	b	adresse 7 bits << 1
54	b	définition du DMA
55	b	l'IT one-shot doit être réarmée
56	b	Bus Fault Address Register

Barème indicatif par module

00 : Q1-10 · 01 : Q11-20 · 02/03 : Q21-30 · 04 : Q31-38 · 07/08 : Q39-50 · 10 : Q51-56. **Un module est validé à 80 %** (ex. 8/10). En dessous : relire la section du cours correspondant à chaque erreur (le corrigé te

donne le mot-clé), refaire le TD, retenter à 48 h.